

Investigation of Range Inertial Graph Based SLAM Algorithms for the European Rover Challenge (2025)

Ángel Aarón Flores Alberca
Universidad Nacional de Ingeniería
Facultad de ciencias
Escuela profesional de Ciencias de la Computación
a.flores.a@uni.pe

Abstract—This paper presents a comprehensive investigation of Range Inertial Simultaneous Localization and Mapping (SLAM) algorithms for potential application in the European Rover Challenge (ERC) 2025. We provide a theoretical foundation starting with probability theory and factor graphs, followed by mathematical approaches to solve Maximum A Posteriori (MAP) inference problems. The paper addresses loop closure solutions and explains the connection between SLAM algorithms and sparse linear algebra. We analyze the architecture of Range Inertial SLAM algorithms and three state-of-the-art examples: KISS-ICP, CT-ICP, and GLIM, comparing their functionalities, requirements, and performance characteristics. The findings of this investigation aim to guide the selection and implementation of optimal navigation strategies for autonomous rovers in challenging planetary exploration scenarios.

Index Terms—SLAM, factor graphs, probabilistic robotics, range-inertial navigation, autonomous exploration, loop closure

I. INTRODUCTION

Autonomous navigation remains one of the fundamental challenges in planetary rover missions. The ability for a rover to precisely locate itself while simultaneously mapping its surroundings—known as Simultaneous Localization and Mapping (SLAM)—is crucial for successful exploration, especially in environments where external positioning systems like GPS are unavailable.

The European Rover Challenge (ERC) represents one of the most significant competitions for university-level rover design teams worldwide. The 2025 competition will likely continue to push the boundaries of autonomous navigation capabilities, requiring robust and efficient SLAM algorithms that can operate in GPS-denied, visually challenging environments with limited computational resources.

Range-Inertial SLAM algorithms, which fuse data from range sensors (like LiDAR) with inertial measurement units (IMUs), have emerged as particularly promising for rover applications due to their robustness in varied lighting conditions and terrain types. This approach leverages the complementary nature of these sensors—LiDAR provides accurate geometric information about the environment while IMUs offer high-frequency motion estimates that help bridge gaps in LiDAR data and provide orientation information.

This paper investigates three cutting-edge Range-Inertial SLAM algorithms: KISS-ICP (Keep It Small and Simple - Iterative Closest Point), CT-ICP (Continuous-Time Iterative Closest Point), and GLIM. Each algorithm represents different approaches to the SLAM problem, with varying trade-offs in terms of computational efficiency, accuracy, and robustness.

We begin by providing a theoretical foundation in probability theory and factor graphs, which form the mathematical backbone of modern SLAM algorithms. We then explore mathematical approaches to solve the Maximum A Posteriori (MAP) inference problem that underpins SLAM. Following this, we address the significant challenge of loop closure detection and correction, which is essential for maintaining consistent maps over longer trajectories. We also explain the deep connections between SLAM algorithms and sparse linear algebra, which enables efficient computational solutions.

The findings of this investigation will inform design decisions for autonomous navigation systems in future rover competitions and may have broader implications for real-world planetary exploration missions, where reliable navigation is paramount for mission success.

II. PROBABILITY THEORY AND FACTOR GRAPHS

A. Probabilistic modeling in SLAM

The SLAM problem is inherently uncertain due to noisy sensor measurements and imperfect motion models. Probability theory provides a principled framework to represent and reason about these uncertainties. In probabilistic SLAM, we aim to estimate the posterior probability distribution over robot poses and landmark positions given sensor measurements.

Let $X = \{x_1, x_2, \dots, x_T\}$ represent the set of robot poses over time and $L = \{l_1, l_2, \dots, l_N\}$ denote the set of landmark positions in the environment. Given a set of measurements $Z = \{z_1, z_2, \dots, z_M\}$, which could include odometry, inertial, and range measurements, the SLAM problem seeks to estimate the posterior probability:

$$p(X, L|Z) \quad (1)$$

Using Bayes' rule, this posterior can be expressed as:

$$p(X, L|Z) = \frac{p(Z|X, L) \cdot p(X, L)}{p(Z)} \quad (2)$$

Since $p(Z)$ is a normalizing constant independent of X and L , the maximum a posteriori (MAP) estimate focuses on maximizing:

$$p(Z|X, L) \cdot p(X, L) \quad (3)$$

We recognize that this formulation, while mathematically sound, presents computational challenges and errors when implemented within traditional Bayesian networks. The optimization structure of this problem—finding the maximum of a product of prior and likelihood terms—suggests a more natural representation.

This motivates the introduction of factor graphs in the following section.

B. Introduction to factor graphs

Factor graphs provide an elegant graphical representation of factorized probability distributions, making them particularly well-suited for SLAM problems. A factor graph is a bipartite graph with two types of nodes: variable nodes representing the unknown states (robot poses and landmark positions) and factor nodes representing probabilistic constraints between variables (derived from measurements).

Formally, a factor graph $G = (V, F, E)$ consists of:

- V : a set of variable nodes (robot poses and landmarks)
- F : a set of factor nodes (measurement constraints)
- E : a set of edges connecting variable nodes to factor nodes

Each factor node $f_i \in F$ represents a function $\phi_i(X_i)$ over a subset of variables $X_i \subseteq V$. The joint probability distribution is then factorized as:

$$p(X, L) \propto \prod_i \phi_i(X_i) \quad (4)$$

In the context of SLAM, these factors typically arise from:

- Motion models: factors between consecutive poses (e.g., from odometry or IMU)
- Measurement models: factors relating poses to landmarks (e.g., from range sensors)
- Loop closure constraints: factors between non-consecutive poses (e.g., from place recognition)
- Prior information: factors on individual variables (e.g., initial pose or known landmark positions)

C. Gaussian factors in SLAM

For the sake of simplicity, let's assume Gaussian noise models that lead to factors of the form:

$$\phi_i(X_i) \propto \exp\left(-\frac{1}{2}\|h_i(X_i) - z_i\|_{\Sigma_i}^2\right) \quad (5)$$

where:

- $h_i(X_i)$ is a measurement prediction function
- z_i is the actual measurement

- Σ_i is the measurement covariance matrix
- $\|e\|_{\Sigma}^2 = e^T \Sigma^{-1} e$ is the squared Mahalanobis distance

The factor graph representation is particularly powerful because it can adapt factor nodes with different distributions and makes explicit the sparse structure of the SLAM problem—each measurement typically involves only a small subset of variables. This sparsity is crucial for developing efficient computational solutions, as we will explore in later sections.

III. MATHEMATICAL APPROACH TO SOLVE A MAP INFERENCE

A. MAP Inference as nonlinear optimization

The Maximum A Posteriori (MAP) estimate in SLAM aims to find the state variables (poses and landmarks) that maximize the posterior probability given the measurements. With Gaussian factors, taking the negative logarithm of the posterior probability transforms the MAP problem into a nonlinear least-squares optimization:

$$\begin{aligned} \{X^*, L^*\} &= \arg \max_{X, L} \prod_i \phi_i(X_i) \\ &\propto \arg \max_{X, L} \prod_i \exp\left(-\frac{1}{2}\|h_i(X_i) - z_i\|_{\Sigma_i}^2\right) \\ &\propto \arg \max_{X, L} \exp\left(\sum_i -\frac{1}{2}\|h_i(X_i) - z_i\|_{\Sigma_i}^2\right) \\ \{X^*, L^*\} &\propto \arg \min_{X, L} \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2 \end{aligned} \quad (6)$$

Thanks to factor graphs, the SLAM problem can be transformed into an optimization one. To approximate a solution, it's important to take a look into linearization for the functions h_i and their implications.

B. Linearization

The standard approach to solving the nonlinear least-squares problem is to iteratively linearize the measurement functions around the current estimate and solve the resulting linear system. Let's denote the state vector as $\theta = [X, L]^T$ and the current estimate as $\theta^{(k)}$. The measurement function can be linearized using a first-order Taylor expansion:

$$h_i(\theta) \approx h_i(\theta^{(k)}) + H_i(\theta - \theta^{(k)}) \quad (7)$$

where $H_i = \frac{\partial h_i}{\partial \theta}|_{\theta^{(k)}}$ is the Jacobian matrix evaluated at the current estimate.

Defining the residual as $r_i = z_i - h_i(\theta^{(k)})$ and the update step as $\delta = \theta - \theta^{(k)}$, the linearized least-squares problem becomes:

$$\delta^* = \arg \min_{\delta} \sum_i \|H_i \delta - r_i\|_{\Sigma_i}^2 \quad (8)$$

This can be more compactly written in matrix form by stacking all Jacobians, residuals, and covariances:

$$\delta^* = \arg \min_{\delta} \|H \delta - r\|_{\Sigma}^2 \quad (9)$$

Based on the above definition, the expression can be rewritten as follows:

$$\begin{aligned}\|H\delta - r\|_{\Sigma}^2 &= (H\delta - r)^T \Sigma^{-1} (H\delta - r) \\ &= (H\delta - r)^T (\Sigma^{-1} H\delta - \Sigma^{-1} r) \\ &= (H\delta)^T \Sigma^{-1} H\delta - (H\delta)^T \Sigma^{-1} r - r^T \Sigma^{-1} H\delta + r^T \Sigma^{-1} r\end{aligned}$$

To obtain a vector δ that minimizes the objective function is necessary to calculate the gradient with respect of δ .

$$\begin{aligned}\nabla \|H\delta - r\|_{\Sigma}^2 &= \nabla ((H\delta)^T \Sigma^{-1} H\delta) - \nabla ((H\delta)^T \Sigma^{-1} r) \\ &\quad - \nabla (r^T \Sigma^{-1} H\delta) + \nabla (r^T \Sigma^{-1} r) \\ &= 2H^T \Sigma^{-1} H\delta - 2H^T \Sigma^{-1} r\end{aligned}$$

Finally equalize the gradient to 0 and obtain the following:

$$\begin{aligned}2H^T \Sigma^{-1} H\delta - 2H^T \Sigma^{-1} r &= 0 \\ (H^T \Sigma^{-1} H)\delta &= H^T \Sigma^{-1} r\end{aligned}\quad (10)$$

This system can be written more compactly as:

$$J\delta = b \quad (11)$$

where $J = H^T \Sigma^{-1} H$ is the information matrix (also called the Hessian) and $b = H^T \Sigma^{-1} r$.

With this, the problem of minimize the sum of squared errors can be transformed to a system of linear equations for which there are several methods to solve.

C. Steepest Descent

By defining an objective function like $\|H\delta - r\|_{\Sigma}^2$, the steepest descent method iteratively updates the SLAM state estimate by repeatedly moving to the direction of the negative gradient of its current estimation:

$$\begin{aligned}\nabla \|H\delta - r\|_{\Sigma}^2(\theta^{(k)}) &= 2H^T \Sigma^{-1} H \times 0 - 2H^T \Sigma^{-1} r \\ &= -2H^T \Sigma^{-1} r\end{aligned}$$

It's also accompanied by a rate of convergence $\alpha \in (0, 1)$ that can be adjusted to determine the percentage of the gradient to be updated.

It's possible to define the next algorithm:

Algorithm 1 Steepest descent for SLAM

- 1: Initialize $\theta^{(0)}, \alpha$
 - 2: **for** $k = 0, 1, 2, \dots$ until convergence **do**
 - 3: Compute residuals $r_i = z_i - h_i(\theta^{(k)})$ for all factors
 - 4: Compute Jacobians $H_i = \frac{\partial h_i}{\partial \theta} |_{\theta^{(k)}}$ for all factors
 - 5: Form the gradient of the objective function at the current state $-2H^T \Sigma^{-1} r$
 - 6: Update $\theta^{(k+1)} = \theta^{(k)} + 2\alpha H^T \Sigma^{-1} r$
 - 7: **end for**
 - 8: **return** $\theta^{(k)}$
-

The downside of this current algorithm is its low convergence rate at finding minimum values.

D. Gauss-Newton Method

The Gauss-Newton algorithm iteratively solves normal equations from the given objective function defined as $J\delta = b$ and updates the state estimate.

The following algorithm is stated:

Algorithm 2 Gauss-Newton for SLAM

- 1: Initialize $\theta^{(0)}$
 - 2: **for** $k = 0, 1, 2, \dots$ until convergence **do**
 - 3: Compute residuals $r_i = z_i - h_i(\theta^{(k)})$ for all factors
 - 4: Compute Jacobians $H_i = \frac{\partial h_i}{\partial \theta} |_{\theta^{(k)}}$ for all factors
 - 5: Form the information matrix $J = H^T \Sigma^{-1} H$ and vector $b = H^T \Sigma^{-1} r$
 - 6: Solve $J\delta = b$ for δ
 - 7: Update $\theta^{(k+1)} = \theta^{(k)} + \delta$
 - 8: **end for**
 - 9: **return** $\theta^{(k)}$
-

E. Levenberg-Marquardt Algorithm

The Levenberg-Marquardt algorithm extends Gauss-Newton by adding a damping term to ensure numerical stability and better convergence:

$$(J + \lambda I)\delta = b \quad (12)$$

where λ is a damping parameter that is adjusted based on the progress of the optimization. When λ is large, the algorithm behaves like gradient descent, providing robustness far from the minimum. When λ is small, it behaves like Gauss-Newton, providing fast convergence near the minimum.

Marquardt proposed to take on count the scaling of the diagonal entries of the information matrix J in order to accelerate converge, taking large steps when the gradient is small and viceversa.

This algorithm can be expressed as:

Algorithm 3 Levenberg-Marquardt for SLAM

- 1: Initialize $\theta^{(0)}$
 - 2: **for** $k = 0, 1, 2, \dots$ until convergence **do**
 - 3: Compute residuals $r_i = z_i - h_i(\theta^{(k)})$ for all factors
 - 4: Compute Jacobians $H_i = \frac{\partial h_i}{\partial \theta} |_{\theta^{(k)}}$ for all factors
 - 5: Form the information matrix $J = H^T \Sigma^{-1} H$ and vector $b = H^T \Sigma^{-1} r$
 - 6: Compute the value $\lambda = \text{diag}(J)$
 - 7: Solve $(J + \lambda I)\delta = b$ for δ
 - 8: Update $\theta^{(k+1)} = \theta^{(k)} + \delta$
 - 9: **end for**
 - 10: **return** $\theta^{(k)}$
-

These methods are particularly valuable in SLAM due to the significant nonlinearities involved, especially with range measurements and rotational components. They provide a good balance between convergence speed and robustness against poor initial estimates or highly nonlinear measurement functions.

IV. SOLVING LOOP CLOSURES

A. The Loop Closure Problem

Loop closure is one of the most challenging aspects of SLAM, occurring when a robot revisits a previously mapped area. Detecting and correctly incorporating loop closures is crucial for:

- Reducing accumulated drift in the robot's trajectory
- Creating globally consistent maps
- Bounding the uncertainty in the state estimate

The loop closure problem consists of two main components:

- 1) **Loop Detection:** Identifying when the robot has returned to a previously visited location
- 2) **Loop Correction:** Updating the map and trajectory to incorporate the loop closure constraint

B. Loop Closure Detection

In Range-Inertial SLAM, loop closure detection typically relies on recognizing similar geometric structures in range data. Common approaches include:

1) **Scan Matching:** Direct comparison of the current scan with previous scans using registration algorithms like ICP or NDT (Normal Distributions Transform). While computationally intensive for large maps, this approach can be made efficient through indexing structures like KD-trees.

2) **Descriptor-Based Methods:** Transforming range data into compact, rotation-invariant descriptors that can be efficiently compared. Examples include Scan Context, M2DP (Matched Direction-Position), and RING (Rotation Invariant Fast Geometrical). These methods enable more efficient searching through a database of previous observations.

3) **Learning-Based Approaches:** Using deep learning to extract features from range data that are invariant to viewpoint changes and can be matched efficiently. Methods like PointNetVLAD, OverlapNet and RoITr (Rotation-Invariant Transformers) have shown promising results in place recognition for 3D LiDAR data. Las

C. Loop Closure Correction

Once a loop closure is detected, the SLAM system must update its estimates to incorporate this new constraint. In the factor graph framework, this is handled elegantly by adding a new factor between the current pose and the matched historical pose.

1) **Pose Graph Optimization:** For pose-graph SLAM, loop closure correction involves:

- Adding a relative pose constraint between the current pose and the matched historical pose
- Re-optimizing the entire pose graph to distribute the accumulated error across the trajectory

The added constraint typically takes the form:

$$\phi_{loop}(x_i, x_j) \propto \exp\left(-\frac{1}{2}\|f(x_i, x_j) - z_{ij}\|_{\Sigma_{ij}}^2\right) \quad (13)$$

where $f(x_i, x_j)$ computes the relative transformation between poses x_i and x_j , and z_{ij} is the measured relative transformation from scan matching.

2) **Incremental Approaches:** Re-optimizing the entire pose graph after each loop closure can be computationally expensive for large maps. Incremental approaches, such as iSAM2 (incremental Smoothing and Mapping), update only the portion of the graph affected by the loop closure, greatly improving efficiency.

These methods typically:

- Identify the variables affected by the new loop closure constraint
- Relinearize only the affected factors
- Update the matrix factorization incrementally

3) **Robust Loop Closure:** False loop closures can catastrophically corrupt the map. Robust methods help mitigate this risk:

- Switchable Constraints: Allow the optimizer to "turn off" outlier loop closures
- Robust Cost Functions: Replace the squared error with functions that are less sensitive to outliers, such as Huber or Cauchy loss
- Max-Mixture Models: Represent uncertain loop closures as a mixture of Gaussians

V. RELATION WITH SPARSE LINEAR ALGEBRA

SLAM algorithms and sparse linear algebra share a profound connection that enables efficient solutions to what would otherwise be computationally intractable problems. Understanding this relationship is crucial for implementing effective SLAM systems on resource-constrained platforms like those used in the European Rover Challenge.

A. The Natural Sparsity of SLAM Problems

SLAM problems inherently generate sparse computational structures, which can be exploited for tremendous efficiency gains. This sparsity emerges from the local nature of robot perception:

- Each sensor measurement relates only a small subset of variables—typically a single pose and the observed landmarks, or two consecutive poses in the case of odometry
- At any given moment, a robot can only observe a small portion of its environment
- The robot's motion follows a trajectory, creating a sequential chain of pose variables with primarily local connections

When formulated as a factor graph, this sparsity becomes immediately visible—most variable nodes connect to only a few factor nodes. For example, in a typical SLAM scenario with 1000 poses and 2000 landmarks, each pose might connect to just 2-3 previous poses and 10-20 landmarks, rather than to all 2999 other variables.

When linearized for optimization, this sparse graph structure translates directly to sparse matrices. The Jacobian matrix H will contain mostly zeros, with non-zero elements only where variables participate in factors. Similarly, the information matrix $J = H^T \Sigma^{-1} H$ preserves this sparsity pattern, though with some additional fill-in.

B. Computational Advantages of Sparse Matrix Methods

The computational implications of this sparsity are profound:

- **Storage efficiency:** Instead of storing $O(n^2)$ elements for an $n \times n$ matrix, sparse matrix formats store only non-zero elements and their indices, often reducing storage requirements from quadratic to linear in the problem size
- **Operation efficiency:** Matrix operations can skip computations involving zero elements, drastically reducing the number of arithmetic operations
- **Scalability:** Problems that would be impossible to solve with dense methods become tractable—while a dense approach might be limited to a few hundred variables, sparse methods can handle problems with hundreds of thousands of variables

When solving the linearized system $J\delta = b$, specialized sparse factorization methods like sparse Cholesky or sparse QR factorization can exploit this structure. These methods avoid operations on zero elements and can reduce the computational complexity from $O(n^3)$ for dense factorization to nearly $O(n)$ for certain graph structures common in SLAM.

C. The Critical Impact of Variable Ordering

The efficiency of sparse matrix factorization is dramatically affected by the order in which variables are eliminated during factorization. This ordering determines the amount of “fill-in”—zero elements that become non-zero during factorization:

- **Poor orderings** can cause extensive fill-in, degrading performance to nearly that of dense factorization
- **Good orderings** maintain sparsity throughout the factorization process, preserving the computational advantages

For rovers in the European Rover Challenge, where computational resources are limited, choosing an appropriate ordering strategy can make the difference between real-time performance and computational failure.

Common ordering strategies include:

- **Approximate Minimum Degree (AMD)** and its column-oriented variant COLAMD, which greedily select variables that will cause minimal fill-in
- **Nested Dissection**, which recursively partitions the graph to find separator sets that divide the problem into independent subproblems
- **Domain-specific heuristics**, such as eliminating landmarks before poses (known as the Schur complement trick), which exploits the bipartite structure of landmark-based SLAM

D. Incremental Updates for Real-time Operation

For online SLAM during rover operation, recomputing matrix factorizations from scratch after each new measurement becomes prohibitively expensive. Incremental matrix factorization methods provide an elegant solution:

- When new measurements arrive, only a portion of the matrix factorization needs to be updated

- For odometry measurements affecting only the most recent poses, updates can be performed in nearly constant time
- Even for loop closures affecting larger portions of the graph, incremental methods still outperform batch recomputation

Teams competing in the ERC should prioritize SLAM implementations that effectively exploit sparsity through appropriate variable ordering and incremental updates. This approach will maximize mapping accuracy and localization precision while minimizing computational overhead, providing a significant competitive advantage in autonomous navigation tasks.

VI. RANGE INERTIAL SLAM ALGORITHMS

This section provides a detailed analysis of three state-of-the-art Range Inertial SLAM algorithms: KISS-ICP (Keep It Small and Simple - Iterative Closest Point), CT-ICP (Continuous-Time Iterative Closest Point), and GLIM. We examine their architectural differences, key features, computational requirements, and performance characteristics that make them suitable for different rover application scenarios.

A. KISS-ICP: Keep It Small and Simple - Iterative Closest Point

KISS-ICP represents a minimalist approach to LiDAR odometry that achieves impressive accuracy with minimal computational overhead. As described by their authors, the algorithm follows the philosophy of “keeping it small and simple” while focusing on the core elements that truly matter for accurate point cloud registration.

1) Key Features:

- **Point-to-Point ICP:** Unlike many modern approaches that use more complex metrics like point-to-plane or feature-based matching, KISS-ICP demonstrates that classic point-to-point ICP can achieve excellent results when implemented properly.
- **Adaptive Thresholding:** A key innovation in KISS-ICP is its adaptive threshold scheme for correspondence matching. Rather than using a fixed distance threshold, it dynamically estimates the appropriate threshold based on the robot’s motion, providing robustness across different environments and motion profiles.
- **Motion Compensation:** KISS-ICP allows for a wide compatibility with different motion compensation approaches from assuming a simple constant velocity to adopting more advanced inertial sensors.
- **Sparse Voxel Structure:** The algorithm uses a voxel-based local map representation that supports efficient neighborhood access with constant-time complexity, significantly outperforming traditional KD-tree implementations.
- **Robust Kernel:** The implementation includes a Geman-McClure robust kernel that helps handle outliers and improper correspondences.

2) *Performance Characteristics*: KISS-ICP shows remarkable versatility across different datasets and sensor types. Testing on standard benchmarks like KITTI, KITTI-raw, KITTI-360, KITTI-CARLA, and ParisLuco demonstrates that it achieves state-of-the-art accuracy while maintaining real-time performance. Notably, it performs well even with different LiDAR sensors and on platforms with highly dynamic motion, such as drones and handheld devices.

The algorithm operates at 60-80 ms per scan on a CPU with a single thread, making it suitable for deployment on resource-constrained platforms like those used in rover competitions. Its ability to handle different motion profiles without parameter tuning is particularly valuable for autonomous rovers that must traverse varied terrain.

3) *Limitations*:

- Lacks built-in loop closure mechanisms, focusing primarily on odometry
- May struggle with extremely degenerate environments where insufficient geometric information is available

B. *CT-ICP: Continuous-Time Iterative Closest Point*

CT-ICP extends traditional ICP approaches by introducing a continuous-time trajectory representation that better handles motion distortion in LiDAR scans. This approach is particularly valuable for scenarios with significant sensor motion during scan acquisition.

1) *Key Features*:

- **Continuous-Time Formulation**: CT-ICP models the trajectory as continuous within scans but discontinuous between scans. Each scan is parameterized by two poses (beginning and end), and points within the scan are transformed based on their timestamps through spherical linear interpolation (SLERP).
- **Discontinuity Between Scans**: Unlike other continuous-time methods that enforce trajectory smoothness across scan boundaries, CT-ICP allows discontinuities between consecutive scans. This provides greater robustness to high-frequency motions that cannot be adequately modeled by smooth interpolation.
- **Local Map Representation**: The algorithm maintains a dense point cloud map stored in a sparse voxel structure, with carefully controlled point density to balance detail and efficiency.
- **Robust Profile**: CT-ICP includes a robust profile mechanism that detects difficult cases (e.g., fast orientation changes) and registration failures, applying more conservative parameters or even skipping map updates when necessary.
- **Loop Closure**: The algorithm incorporates a fast, pure LiDAR-based loop detection approach using elevation image matching, enabling global consistency in maps.

2) *Performance Characteristics*: CT-ICP demonstrates excellent performance on standard datasets, achieving an average relative translation error of 0.59% on the KITTI odometry benchmark, making it among the top-performing algorithms with publicly available code. It handles both automotive scenarios and high-frequency motion profiles (drones, handheld devices, segways) robustly with the same parameter set.

The algorithm achieves real-time performance, processing each scan in approximately 60 ms on a CPU with a single thread. When loop closure is enabled, additional processing is required but remains efficient enough for online operation.

3) *Limitations*:

- The loop closure implementation requires the motion to be mostly planar and the gravity vector to be known
- More complex than KISS-ICP, requiring careful implementation
- May struggle in environments with limited geometric features

C. *GLIM: 3D Range-Inertial Localization and Mapping with GPU-Accelerated Scan Matching*

GLIM represents a comprehensive approach to range-inertial SLAM that leverages GPU acceleration to enable sophisticated algorithms to run in real-time. This framework integrates range and inertial data in both odometry estimation and global mapping.

1) *Key Features*:

- **GPU-Accelerated Scan Matching**: GLIM uses GPU parallelization to efficiently compute scan matching constraints, enabling real-time operation despite the computational complexity of its algorithms.
- **Fixed-Lag Smoothing with Keyframes**: The odometry module employs a combination of fixed-lag smoothing and keyframe-based point cloud matching that can handle momentary degeneration of range data while efficiently reducing trajectory drift.
- **Tightly Coupled IMU Integration**: Unlike many approaches that loosely couple IMU data, GLIM tightly integrates inertial measurements in both the odometry and global optimization modules, improving stability and accuracy.
- **Multi-Camera Visual Extension**: The framework supports the integration of visual features from multiple cameras to further enhance estimation accuracy.
- **Global Registration Error Minimization**: Rather than standard pose graph optimization with relative pose constraints, GLIM's global optimization directly minimizes registration errors between submaps across the entire map.
- **Submap Endpoints**: GLIM introduces the concept of submap endpoints to maintain strong IMU constraints between submaps with larger time intervals.

2) *Performance Characteristics*: GLIM demonstrates exceptional robustness to degenerated sensor data, maintaining good trajectory estimates even when range data becomes completely unreliable for short periods. Tests on various datasets show it outperforming state-of-the-art methods like FAST-LIO2 and VoxelMap in challenging scenarios.

The framework also shows remarkable versatility across sensor types, working effectively with spinning LiDARs, non-repetitive scan LiDARs, ToF depth cameras, solid-state LiDARs, and even stereo cameras.

Despite its sophisticated algorithms, GLIM achieves real-time performance through careful GPU optimization. Processing times vary by module, with preprocessing taking around 15 ms per frame, odometry estimation 30 ms, and global mapping 55 ms per submap.

3) Limitations:

- Requires a GPU for real-time performance, potentially limiting deployment on very resource-constrained platforms
- More complex implementation than simpler algorithms like KISS-ICP
- Higher memory requirements due to storage of submaps and optimization structures

D. Comparative Analysis

When considering these algorithms, several factors must be weighed. Table I provides a comparative summary of the three algorithms across key evaluation criteria.

For the competition, the choice between these algorithms would depend on the specific competition requirements, computational resources available on the rover, and the complexity of the environment to be navigated.

- KISS-ICP offers the most lightweight solution, making it ideal for rovers with limited computational resources, while still providing excellent accuracy across various datasets. Its simplicity makes it easier to implement and integrate into existing systems.
- CT-ICP provides a good balance between computational efficiency and capability, with its continuous-time formulation handling motion distortion particularly well. This makes it suitable for general navigation across varied terrain, especially when loop closure is needed.
- GLIM represents the most comprehensive solution, with superior robustness to sensor degradation and native multi-sensor fusion. However, its reliance on GPU acceleration may limit its applicability on platforms with strict power or weight constraints.

TABLE I
COMPARATIVE SUMMARY OF RANGE-INERTIAL SLAM ALGORITHMS

Aspect	KISS-ICP	CT-ICP	GLIM
Compute Needs	Low (CPU only)	Medium (CPU only)	High (GPU required)
Motion Handling	Good	Excellent	Superior
Loop Closure	None native	Yes (planar)	Advanced global
Sensor Fusion	External only	Limited	Tightly integrated

The key distinction between these algorithms lies in their approach to handling sensor data: KISS-ICP emphasizes simplicity with its point-to-point ICP and adaptive thresholding, CT-ICP focuses on continuous-time representation with elastic formulation, while GLIM employs sophisticated optimization techniques with tight sensor integration and GPU acceleration.

VII. CONCLUSIONS

This investigation has provided a comprehensive analysis of Range-Inertial SLAM algorithms in the context of the European Rover Challenge. We established the theoretical foundations through probability theory and factor graphs, which provide a principled framework for representing the SLAM problem. We explored the mathematical approaches to solve MAP inference, highlighting the connection to nonlinear optimization and sparse linear algebra, which enables efficient solutions on resource-constrained platforms.

Our examination of three state-of-the-art algorithms—KISS-ICP, CT-ICP, and GLIM—reveals different approaches to the fundamental challenges in SLAM: handling sensor noise, compensating for motion distortion, maintaining global consistency, and operating with limited computational resources.

KISS-ICP demonstrates that simplicity and careful implementation of fundamentals can outperform more complex approaches. Its minimal parameter set and adaptive threshold scheme make it particularly suitable for rapid deployment across varied environments without extensive tuning. For teams with significant computational constraints, KISS-ICP offers an excellent balance between accuracy and efficiency.

CT-ICP advances the field by addressing the continuous nature of sensor acquisition while acknowledging the discontinuities in real-world motion. Its elastic formulation handles complex motion profiles remarkably well, making it suitable for rovers traversing challenging terrain where sudden movements are common. The addition of efficient loop closure capabilities further enhances its utility for longer missions.

GLIM represents perhaps the most sophisticated approach, leveraging GPU acceleration to enable computations that would otherwise be prohibitively expensive. Its robustness to momentary sensor degradation and ability to incorporate multiple sensor modalities make it ideal for mission-critical applications where failure is not an option. For teams with access to GPU resources and complex navigation requirements, GLIM provides comprehensive capabilities that could provide a competitive edge.

As computational resources on rover platforms continue to improve, more sophisticated approaches will become increasingly feasible. The theoretical foundations and algorithms discussed in this paper provide a solid framework for teams to build upon, potentially enabling new capabilities that push the boundaries of autonomous planetary exploration.

The European Rover Challenge represents an ideal testbed for these technologies, with its combination of structured tasks, unpredictable environments, and competitive pressure driving innovation. By carefully selecting and implementing Range-Inertial SLAM algorithms appropriate to their specific rover platforms, teams can achieve the reliable and accurate navigation essential for mission success.

REFERENCES

- [1] Factor Graphs for Robot Perception. (s. f.-b). Now Foundations And Trends Books — IEEE Xplore. <https://ieeexplore.ieee.org/document/8187520>

- [2] Vizzo, I., Guadagnino, T., Mersch, B., Wiesmann, L., Behley, J., & Stachniss, C. (2023b). KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way. *IEEE Robotics And Automation Letters*, 8(2), 1029-1036. <https://doi.org/10.1109/ra.2023.3236571>
- [3] Dellenbach, P., Deschaud, J., Jacquet, B., & Goulette, F. (2021b, septiembre 27). CT-ICP: Real-time Elastic LiDAR Odometry with Loop Closure. *arXiv.org*. <https://arxiv.org/abs/2109.12979>
- [4] Koide, K., Yokozuka, M., Oishi, S., & Banno, A. (2024b). GLIM: 3D range-inertial localization and mapping with GPU-accelerated scan matching factors. *Robotics And Autonomous Systems*, 179, 104750. <https://doi.org/10.1016/j.robot.2024.104750>
- [5] The MIT Press, Massachusetts Institute of Technology. (2024, 18 junio). Book details - MIT Press. MIT Press. <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>
- [6] Barfoot, T. D. (2017). *State Estimation for Robotics*. <https://doi.org/10.1017/9781316671528>
- [7] Davis, T. A., Rajamanickam, S., & Sid-Lakhdar, W. M. (2016). A survey of direct methods for sparse linear systems. *Acta Numerica*, 25, 383-566. <https://doi.org/10.1017/s0962492916000076>
- [8] A Tutorial on Graph-Based SLAM. (2010, 1 enero). *IEEE Journals & Magazine — IEEE Xplore*. <https://ieeexplore.ieee.org/document/5681215>
- [9] Zhang, J., & Singh, S. (2014). LOAM: Lidar Odometry and Mapping in Real-time. *ResearchGate*. <https://doi.org/10.15607/rss.2014.x.007>
- [10] Shan, T., & Englot, B. (2018). LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain. 2021 *IEEE/RSJ International Conference On Intelligent Robots And Systems (IROS)*. <https://doi.org/10.1109/iros.2018.8594299>
- [11] Shan, T., Englot, B., Meyers, D., Wang, W., Ratti, C., & Rus, D. (2020, 1 julio). LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping. *arXiv.org*. <https://arxiv.org/abs/2007.00258>
- [12] Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I., & Leonard, J. J. (2016). Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions On Robotics*, 32(6), 1309-1332. <https://doi.org/10.1109/tro.2016.2624754>
- [13] Pomerleau, F., Colas, F., & Siegwart, R. (2015). A Review of Point Cloud Registration Algorithms for Mobile Robotics. *Foundations And Trends In Robotics*, 4(1), 1-104. <https://doi.org/10.1561/23000000035>